

Automated Document Review and Task Tracking System Using Python-Based Workflow Automation

BITS SEZG628T: Dissertation

by

V HARSHITHA
(2024TM93089)

Dissertation work carried out at

Accenture, Bangalore

Submitted in partial fulfilment of **MTech in Software
Engineering** degree programme

Under the Supervision of

Priyanka Nagane
Accenture, Pune



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

PILANI (RAJASTHAN)

2024-26

ACKNOWLEDGEMENT

I would like to thank everyone who has helped me to complete this project.

I want to start by thanking my mentor and supervisor Priyanka Nagane for her continued support and helpful suggestions at various points throughout the project and Sandeep Parakunnath for working with me closely to help me to understand the potential business value of this project through the various stages of development as well as providing real-time feedback along the way.

I thank my friends and family for being there for me and having faith in me; your support and encouragement were vital factors in my success.

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
WILP Division
May, 2026

Organization: Accenture

Location: Bengaluru

Duration: 14 weeks

Date of Start: 15th Jan 2026

Date of Submission: 10th May 2026

Title of the Project: Automated Document Review and Task Tracking System Using Python-Based Workflow Automation

Name of the student/ BITS ID: V Harshitha, 2024TM93089

Name and Designation of Supervisor: Priyanka Nagane, Packaged Application Development Team Lead

Name and Designation of Additional Examiner: Sandeep Parakunath, Offshore Delivery Manager

Name of the Faculty mentor: Pravin Yashwant Pawar

Key Words: Workflow Automation, Document Management Systems (DMS), Full-Stack Web Development, Role-Based Access Control (RBAC), Finite State Machine (FSM), RESTful API Architecture, Asynchronous Task Scheduling, Secure Client-Server Architecture.

Project Areas:

- **Primary Research Area:** Software Engineering and Enterprise Application Development.
- **Sub-domains:** Workflow Automation, Full-Stack Web Architecture, and Secure Systems Design.
- **Workflow Automation & State Management:** Document life cycles, Compliance rules and automate time-based transitions
- **Secure Systems & Access Control:** REST API and role-based access control (RBAC) using JWT
- **Full-Stack Web Architecture:** Frameworks such as React.js and FastAPI to create responsive user interfaces, allow for asynchronous data processing and create scalable relational databases- PostgreSQL.
- **Background Processing & Task Scheduling:** Asynchronous scheduler (APScheduler)
- **AI Integration & Document Parsing:** Parsing, Text extraction tools Python (PyPDF2)

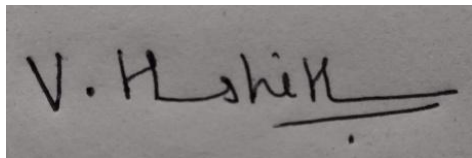
ABSTRACT

The document review/approval process is very important in regulated industries (e.g., life sciences) because these processes help to ensure that all documents are compliant with regulations, correct and have accurate data. Many companies, however, still use manual processes that are prone to error or rigidly configured enterprise software packages that are inflexible for current workflow base needs. These legacy systems

often operate as “black boxes” with little or no visibility of the operation of the automation, customizability or flexibility as workflow evolves. Therefore, software engineers face a number of challenges in modifying the internal behavior of these systems to meet requirements of today's agile development.

This project seeks to address these challenges through the development and design of a secure, Automated Document Workflow and Task Tracking System. This full-stack web application replaces the manual or rigid monitoring of documents with an automated document-monitoring/automated compliance checking engine developed in Python. This implementation provides for the automation of document compliance checks, implementation of strict Role-Based Access Control (RBAC) as a means of ensuring secure document access and maintaining an immutable audit trail as a mechanism for providing full traceability of the use of the Document Workflow and Task Tracking System through an entirely open, configurable automation architecture.

The project demonstrates how a flexible backend architecture combined with a standard web interface allows for an efficient and secure document lifecycle management process, without the need for using closed-design enterprise software. The resultant framework of the project provides an open and adaptable means of automating document workflows that was created from the ground up. Additionally, the lightweight in-process scheduling system replaces complicated distributed task queues of moderate applications with reduced infrastructure overhead towards increasing both the efficiency and reliability of the total system.



Signature of the Student

Name: V Harshitha

Date: 10-05-2026

Place: Bangalore



Signature of the Supervisor

Name: Priyanka Nagane

Date: 10-05-2026

Place: Pune

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Dissertation entitled “**Automated Document Review and Task Tracking System Using Python-Based Workflow Automation**” and submitted by **V Harshitha** having ID no. **2024TM93089** in partial fulfillment of the requirements of Dissertation, embodies the work done by her under my supervision.



Place: Bengaluru
Date: 10th-May-2025

Signature of the Supervisor
Priyanka Nagane

Contents

• CHAPTER 1: INTRODUCTION	8
1.1 Overview	8
1.2 Purpose of the system	8
1.3 Key Objectives	8
• CHAPTER 2: SCOPE OF THE PROJECT	9
2.1 In- Scope Features and capabilities	9
2.2 Out-of-Scope Capabilities	9
• CHAPTER 3: LITERATURE REVIEW	10
3.1 Evolution of Document Management Systems	10
3.2 Traditional Automation and Distributed Queues	10
3.3 The Proposed Approach: In-Process Asynchronous Scheduling.....	10
3.4 Secure Client-Side Document Rendering	11
3.5 Comparative Analysis of Traditional Vs New System	11
• CHAPTER 4: ABOUT PROJECT	11
4.1 MODULES IN DOCUMENT WORKFLOW AUTOMATION.....	11
4.2 FUNCTIONAL ARCHITECTURE DIAGRAM/ DESCRIPTION	13
4.3 DESIGN CONSIDERATIONS	14
• CHAPTER 5: WORKING OF THE APPLICATION	14
5.1 FLOW DIAGRAM	14
5.2 SEQUENCE DIAGRAM	15
• CHAPTER 6: RESULTS	16
6.1 Registration Page.....	16
6.2 Login Page.....	16
6.3 Dashboard Page	17
6.4 Documents Page.....	17
6.5 Document Details Page.....	18
6.6 My Tasks Page	19
6.7 Audit Logs Page.....	19
6.8 Notifications Tab	20
6.9 System Performance and Observations	20
• CHAPTER 7: TESTING	21
7.1 Testing Approach.....	21
7.2 Sample Test Cases.....	21
7.3 Validation	21

• CHAPTER 8: CONCLUSIONS	22
8.1 Summary	22
8.2 Future Scope	22
8.3 Challenges Faced	23
• REFERENCES	24
• ABBREVIATIONS	24
• APPENDIX	24

Figures

Figure 1: FUNCTIONAL ARCHITECTURE DIAGRAM	13
Figure 2: FLOW DIAGRAM	14
Figure 3: SEQUENCE DIAGRAM	15

Tables

Table 1: Comparative Analysis of Traditional Vs New System	11
Table 2: MODULES IN DOCUMENT WORKFLOW AUTOMATION	12
Table 3: Sample Test Cases	21

CHAPTER 1: INTRODUCTION

1.1 Overview

In the modern life sciences industry, managing the lifecycle of a document from initial draft to final approval is often an inefficient, manual process. Organizations still rely on traditional communication channels such as emails and basic file servers which frequently cause missed deadlines, version confusion and poorly managed task assignments. When an organization does not have an automated tracking system for day-to-day administrative activities, these workflows can be highly inefficient by creating a bottleneck in data quality and overall operational efficiency.

Therefore, there is an obvious need for a customizable, transparent and automated workflow system that will evolve with the organization's needs without requiring a strict enterprise solution.

1.2 Purpose of the System

The Document Workflow Automation System is a full-stack software application developed to remove administrative bottlenecks by implementing a time-based automated workflow system. This system will create a central secure location that reduces human involvement in tracking deadlines and assigning tasks. By using a modern technology stack, this software will provide a reliable, transparent and efficient workflow for both document authors and reviewers.

1.3 Key Objectives

- To develop a RESTful architecture for secure file handling and user authentication to be developed using JWT technology.
- To design and implement an automated workflow with clear states for reviewing documents will be implemented.
- To develop a custom workflow management system that will allow for transitions between the different states of a document's life cycle dependent upon the user's role and actions.
- To automate all aspects of task creation, assignment and completion in the document review and approval process.
- To create an audit history of all system activity.
- To create a background process that monitors document deadlines and

automatically triggers transitions based on state changes without user intervention.

- To develop an in-browser document viewer that allows reviewers to view PDF documents without downloading the document locally.
- To develop an interface that includes document management capabilities, task tracking capabilities and workflow visualization capabilities in an intuitive manner.

CHAPTER 2: SCOPE OF PROJECT

2.1 In-Scope Features and Capabilities

This project describes the process of developing an integrated client/server application that includes both client and server, the two components function independently, while also being able to communicate with each other seamlessly. It includes the following key features of the Application:

- **Automated State Management:** The Application will automatically manage the status of the document (from Draft to Pending Review) after it has reached its deadline, without requiring any manual intervention.
- **Automated Version Control:** The Application will maintain a record of all versions of a document. When a Document Author uploads a corrected document, the Application will assign a new version to it automatically (i.e., from v1.0 to v2.0), so that Document Reviewers will be able to identify the number of revisions made to a document during the process.
- **Role-Based Task Assignment:** The System will assign all reviewers automatically when the document is ready for review.
- **Automated Text Summarization:** Integration of the PyPDF2 library to generate AI-generated summaries by extracting text from PDFs up to 3,000 words.
- **Secure Document Rendering:** Users can display documents securely as they are opened in the application (pop-up/modal). Only PDF files can be displayed securely by the system.
- **Real-time Handling:** The System uses Local Time (IST), so when a user inputs a deadline, it directly corresponds to what the back-end processes.

2.2 Out-of-Scope Limitations

To keep the system simple and lightweight, some advanced features are intentionally not included:

- **Dedicated Message Brokers:**

The project does not use tools like Celery, Redis or RabbitMQ. Instead, the background processing runs along with the main server, making the setup simpler but less suitable for very large-scale systems.

- **Multi-Format Document Rendering:** The document viewer built into the system will only support viewing of PDF files. Neither Word (.docx), nor Excel (.xlsx) files can be viewed directly in the system, they must first be downloaded and then opened.

CHAPTER 3: LITERATURE REVIEW

3.1 Evolution of Document Management Systems

Earlier, document management systems mainly worked as simple storage platforms like FTP servers or basic cloud tools such as Google Drive or Dropbox. These systems helped store and organize files digitally but they did not support automation.

Users had to manually check for new documents or send emails to reviewers, especially when deadlines were near. This resulted in slower workflows and higher dependency on manual intervention.

3.2 Traditional Automation and Distributed Queues

In large enterprise systems like SAP or Microsoft SharePoint, automation is utilized differently. These systems separate the main web application from background tasks. To perform automation, many enterprises utilize a messaging service such as RabbitMQ and Celery. The advantage of this architecture is that it is flexible and can scale (depending on the processing speed of the servers involved). The disadvantage of this architecture is the complexity and use of several different servers making the deployment and management of the resulting application more difficult.

3.3 The Proposed Approach: In-Process Asynchronous Scheduling

Modern frameworks for building web apps with Python make it easier to manage background tasks. FastAPI can handle high amounts of traffic/requests at once because of FastAPI's ability to be built on top of other technologies, such as Starlette and ASGI.

In this project, instead of using complex external systems for background processing, a lightweight approach is used. APScheduler is integrated directly into the application that runs inside the same server.

Every minute (or 60 seconds), APScheduler checks for documents that are past their due

date and updates the document status. This method automates some aspects of large enterprise systems while using a much simpler and more manageable architecture.

3.4 Secure Client-Side Document Rendering

Additionally, to provide a greater level of security, the frontend uses more secure methods for displaying documents on the client (e.g. using the HTTP fetch API with a temporary Blob URL) as opposed to traditional methods of displaying documents in an iframe (which can be less secure).

3.5 Comparative Analysis of Traditional Vs New System:

The observable difference between a standard document management system and the new proposed system, specifically when considering flexibility, automation and transparency.

Feature	Traditional Systems	Proposed System
Workflow Automation	Limited / Manual	Fully Automated
Customization	Restricted	Highly Flexible
Transparency	Low (Black-box systems)	High (Audit logs)
Infrastructure Complexity	High	Moderate
Scalability	High (with cost)	Moderate (extendable)

Table 1: Comparative Analysis of Traditional Vs New System

CHAPTER 4: ABOUT PROJECT

4.1 MODULES IN DOCUMENT WORKFLOW AUTOMATION

The main modules in Document Workflow Automation are namely users, documents, tasks, audit and notifications.

Module	Description
Users	It manages registration, login and role-based access control.
Documents	It manages file metadata and status of the documents (Draft, Pending, Approved).

Tasks	It automatically creates review tasks and assigns them to specific users once a document is submitted.
Audit	It's a read-only tracker that logs every transaction, including who did what, when and on which document.
Notifications	This module sends real-time alerts to users about task assignments and approval or rejection outcomes.

Table 2: MODULES IN DOCUMENT WORKFLOW AUTOMATION

- a) **Users Module:** The Users Module contains information about users and their roles such as ID, username, email, password and the ability to perform actions in the application based on their role as Author or Reviewer or both.
- b) **Documents Module:** The Documents Module is where documents will be created and stored. This includes storing the document ID, document title, document content/metadata in PDF format, who created the document (the document owner), AI-generated summaries and the current status of the document (draft, pending, approved, rejected). It will also track the life cycle of a document and maintain its status throughout the workflow.
- c) **Tasks Module:** The Tasks Module manages the task creation and assignment based on document submissions. When a document is submitted for review, a task will be automatically created and assigned to a reviewer. Review tasks include task ID, document ID, reviewer, task status and the time the task was created.
- d) **Audit Module:** The Audit Module is an audit log of all the activities performed within the application. Each log entry contains the following information: user ID, document ID, action performed (create, update, approve, reject), timestamp. This module promotes transparency, traceability and compliance.
- e) **Notifications Module:** The Notifications Module is to provide the system with real-time alerts. Alerts include the information regarding tasks assigned to users, document status changes (approved/rejected) and other updates that may occur. Notification information will include: notification ID, user ID, message.

4.2 FUNCTIONAL ARCHITECTURE DIAGRAM/ DESCRIPTION:

- **Architectural Approach:** The system is designed in a way where the frontend and backend are separate. They communicate with each other using REST APIs, which makes the application more flexible and easier to manage.
- **Backend Engine:** The backend is built using FastAPI because it is fast and supports asynchronous operations. It also uses Pydantic to validate data and ensure everything runs smoothly.
- **Database Design:** A relational database, PostgreSQL, is used along with SQLAlchemy. The database is organized into separate tables for users, documents, tasks, and audit logs to keep things clean and modular.
- **Frontend Framework:** A Dynamic Responsive UI has been built using React.js. The UI will change depending on the User's Role and will be built with secure JWT Authentication.
- **Workflow Logic:** The system follows a defined workflow lifecycle (explained in Chapter 5) and supports version tracking for updates.

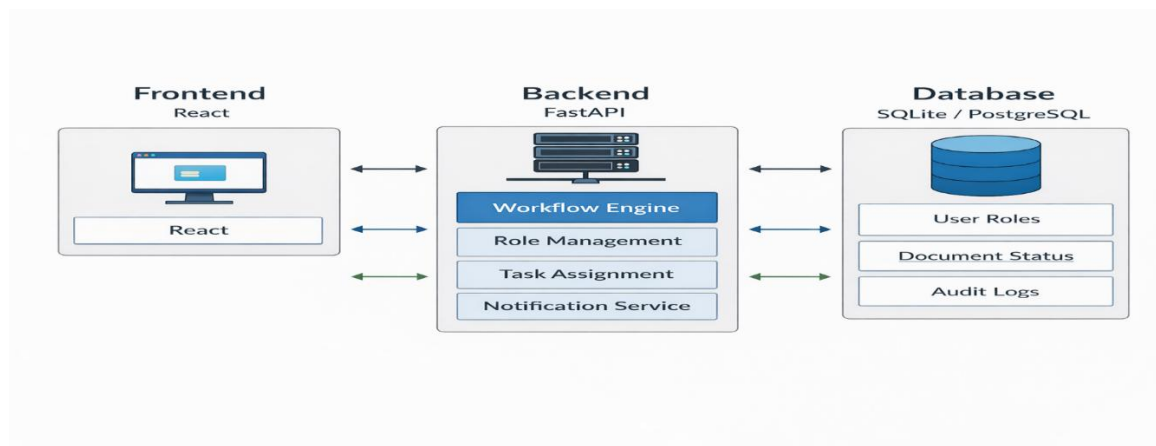


Figure 1: FUNCTIONAL ARCHITECTURE DIAGRAM/ DESCRIPTION

The React frontend communicates with the FastAPI backend via REST APIs using the request-response cycle, with POST requests sent from the client to the server. The FastAPI validates the payload using Pydantic models and runs business logic (authenticating users and transitioning workflows) before saving data in PostgreSQL. To monitor deadlines and initiate automated state transitions, a background task is scheduled using APScheduler. These processes ensure consistency between user actions and automated actions taken

by the system.

4.3 DESIGN CONSIDERATIONS:

- **State Machine Logic:** A document cannot move to "Approved" unless a task is explicitly approved by a user with the "Reviewer" role to ensure workflow reliability,
- **Circular Workflow:** If a document is rejected, the system automatically reverts the state to "Draft" allowing the Author to revise and re-submit the document, creating a complete lifecycle loop.
- **Security:** API endpoints are protected based on user roles to prevent Authors from self-approving their own documents and the restricted document content is secured for confidential purposes.

CHAPTER 5: WORKING OF THE APPLICATION

5.1 FLOW DIAGRAM

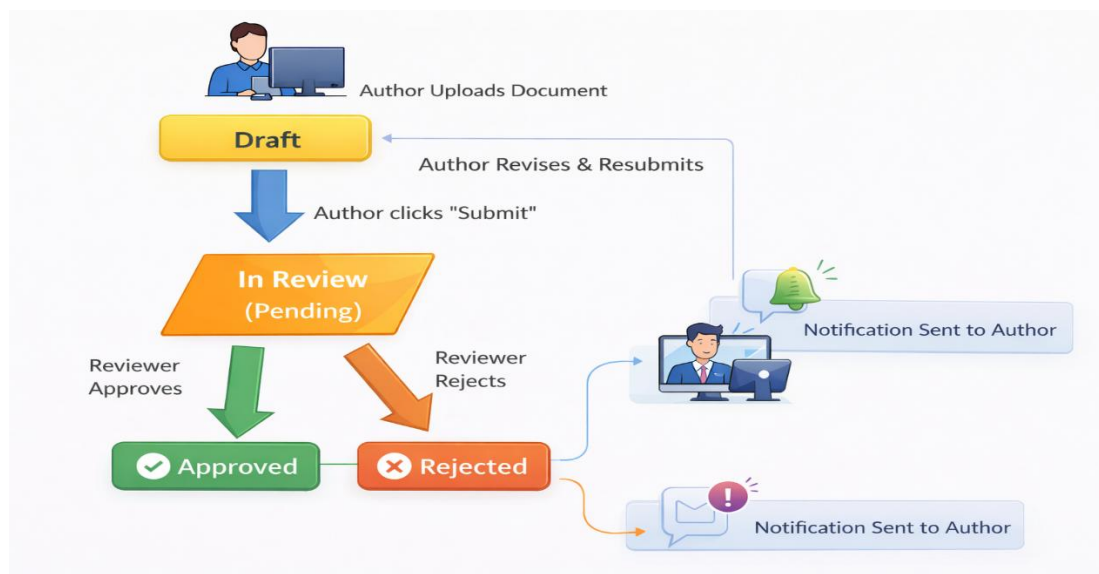


Figure 2: Flow Diagram

The diagram illustrates how a document moves through its lifecycle within the system from creation through to approval or rejection. The workflow begins with an Author uploading a new document and creating a draft version of the document. The Draft file is securely stored by the system and given draft status. Throughout this initial phase of the document's lifecycle, the Author has full edit privileges, allowing the author to continue making changes to the document, and/or to upload subsequent versions as desired. After finalizing the document, the Author submits the document which triggers a state change, and thus a

change from Draft status to In Review status (Pending). At this stage in the document lifecycle, the document will be locked (i.e., no further authorial edits) and it will now transition from the Draft phase into the Review/ Assessment phase. The document will now be evaluated by the designated Reviewer who will determine if the document passes all compliance requirements. If the document passes all compliance requirements, the Reviewer will authorize the document and transition it into the terminal Approved state, completing the document lifecycle.

5.2 SEQUENCE DIAGRAM

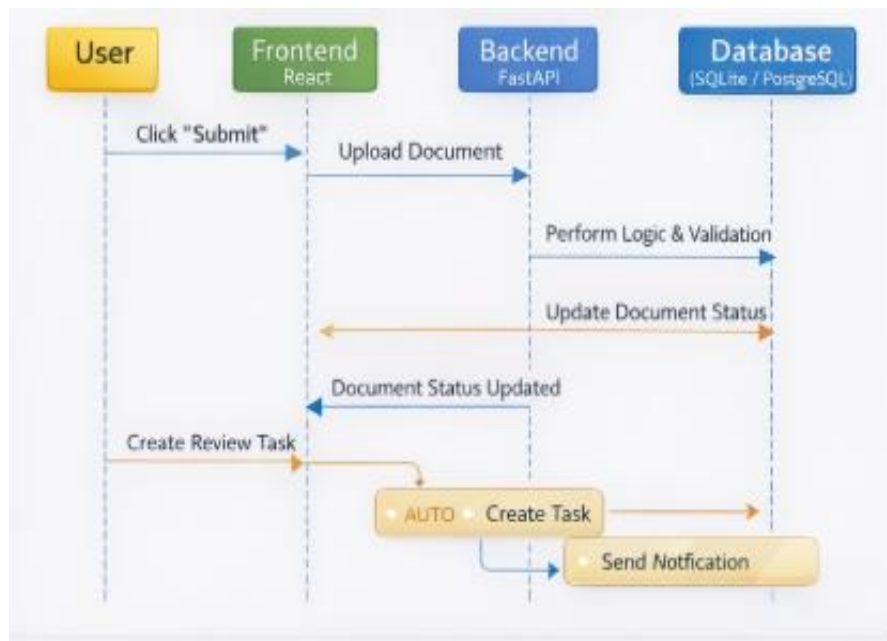


Figure 3: Sequence Diagram

Client Request: After clicking "Submit" on the user interface (UI), the React front-end dispatches a secure REST API request to the FastAPI back-end to process it.

Authentication & Validation: The back-end processes the secure request and executes strict business logic to validate the incoming request and JWT from the user before processing it.

State Mutation: After all validations have passed, the back-end will communicate with a PostgreSQL database to update the record and change its status to "In Review".

Client Acknowledgement: The back-end will return a success response of (HTTP 200 OK) to the front-end immediately so that the UI can update and inform the user that the request has been submitted.

Automated Task Delegation: After updating the database, the back-end creates a new review by automatically assigning a reviewer from the database according to their ID.

Notification Dispatch: The system also triggers the notification module to notify the assigned reviewer of the pending task and to notify the author that the workflow has begun.

This series of steps allows for the controlled and secure performance of all actions with distinct separation between client interaction, backend processing and database updates. Additionally, automated notifications and task delegation increase the ability of the system to respond without requiring any manual intervention.

CHAPTER 6: RESULTS

6.1 Registration Page

New users can gain access into the application by creating an account from the Registration Page where they enter standard registration information like full name, e-mail address and a unique password. The most important information for a user to complete on this page is the Role Selection. Users can register as an Author (to upload documents), a Reviewer (to view and evaluate documents) or Both (to have hybrid roles allowing the user to create and approve documents from one account without the need for profile switching.)

← ↻ 📍 localhost:5173/register Summarize 🔍 ⌕ ⭐ ⚙

Create an account

[Or sign in to your existing account](#)

Full Name
👤 Enter your Full Name

Email address
✉ your@mail.com

Password
🔒

Select Your Roles

☒ **Author**
I want to submit documents for review.

☐ **Reviewer**
I want to review and approve documents.

Create Account →

6.2 Login Page

After registering, users can sign in to the application via the Login page. To make the process simpler, the system automatically creates usernames based on the user's email address, using the text right before the '@' symbol. By entering this generated username and their password, users gain secure access to the system.

localhost:5173/login



Sign in to your account

Document Management System

Or [Register for a new account](#)

Username

Password

[Sign In](#)

6.3 Dashboard Page

Once logged in, users will be directed to the dashboard. This page provides a clear overview of the application's overall progress. It highlights the total number of documents, how many are pending and how many are approved. It also includes a list of recently added documents for easy monitoring.

Document Management System

Dashboard Documents My Tasks Audit Log

Welcome, [Author_harshitha](#) [Logout](#)

Dashboard Overview



Total Documents

4



Pending Review

0



Approved

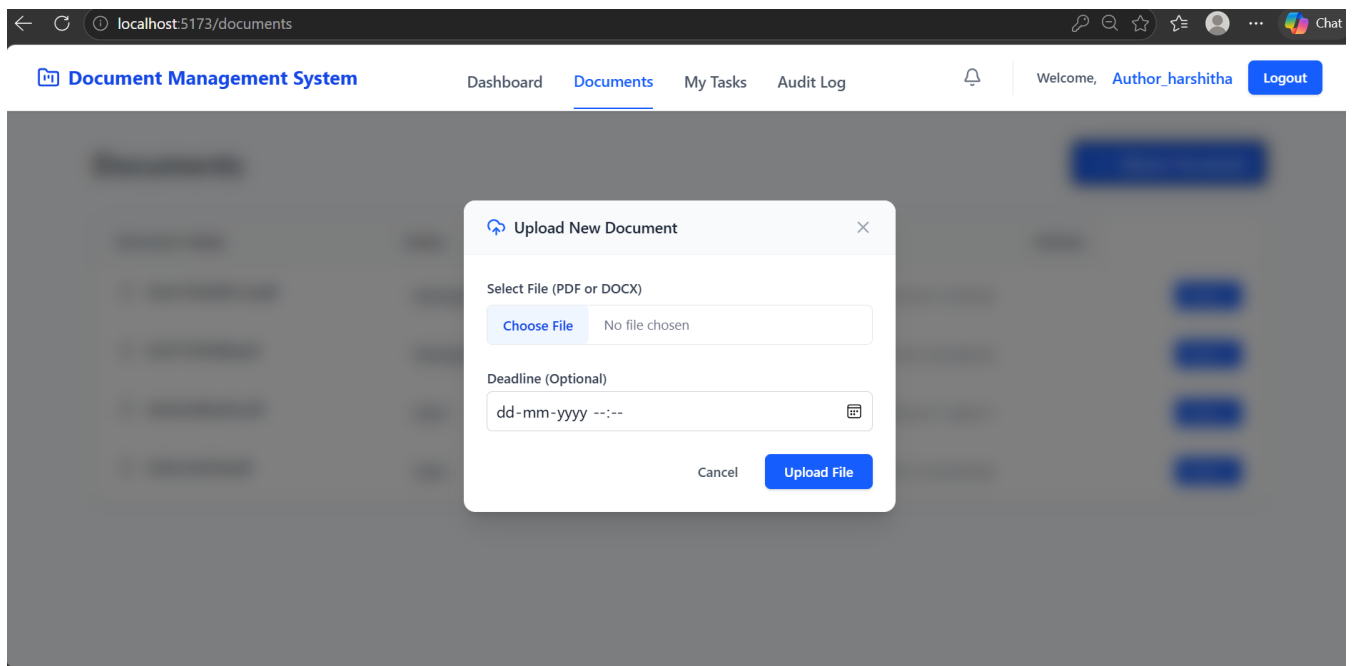
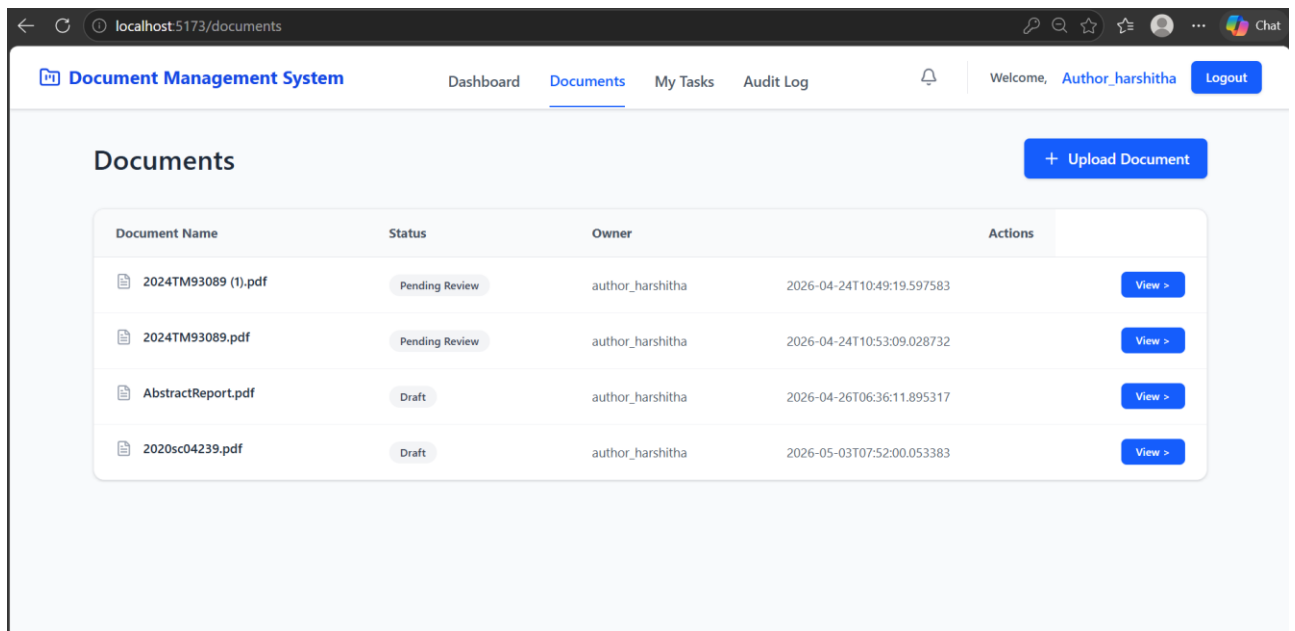
0

Recently Added Documents

 2020sc04239.pdf	DRAFT	Review
 AbstractReport.pdf	DRAFT	Review
 2024TM93089.pdf	PENDING REVIEW	Review
 2024TM93089 (1).pdf	PENDING REVIEW	Review

6.4 Documents Page

The Document Page serves as the central repository for all the documents stored in the system. It provides a visible directory of all files however only Authors can upload new files. When an Author uploads a new file, they have the option to set a deadline for the review process.

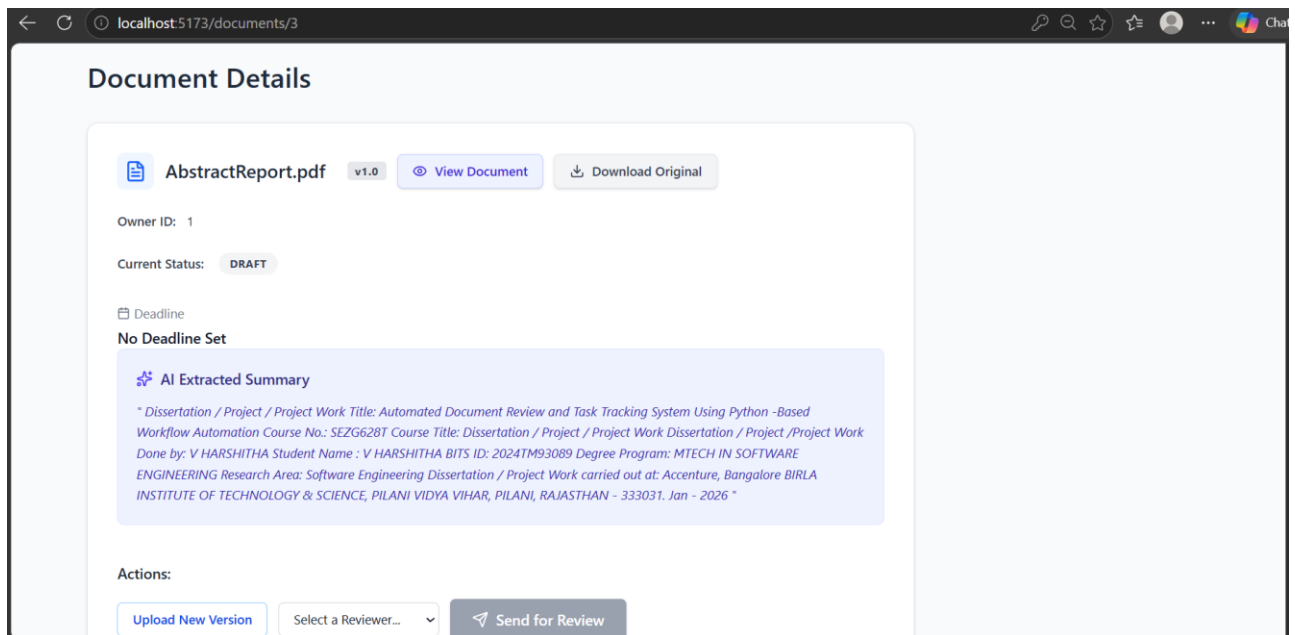


6.5 Document Details Page

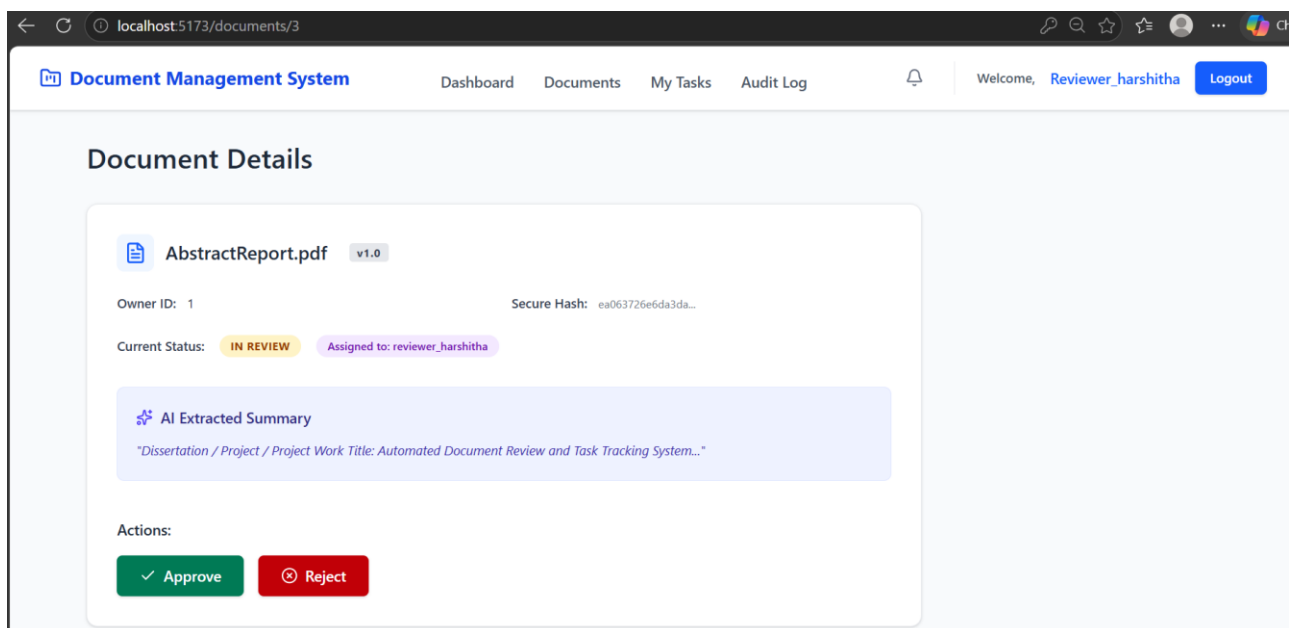
Users can view all information related to the selected document from the Document Details page. It displays the file's name, version, current status and an AI-generated summary. Users can choose to preview the document directly in the browser or download the document.

The Actions section for authors gives authors the ability to upload a new version as well as assign the document for manual review to any reviewer in the system. Additionally, the system monitors the document for a deadline when the deadline is passed and no new version has been uploaded, the system will automatically send the document to all assigned

reviewers so that the document moves through the workflow without delay.



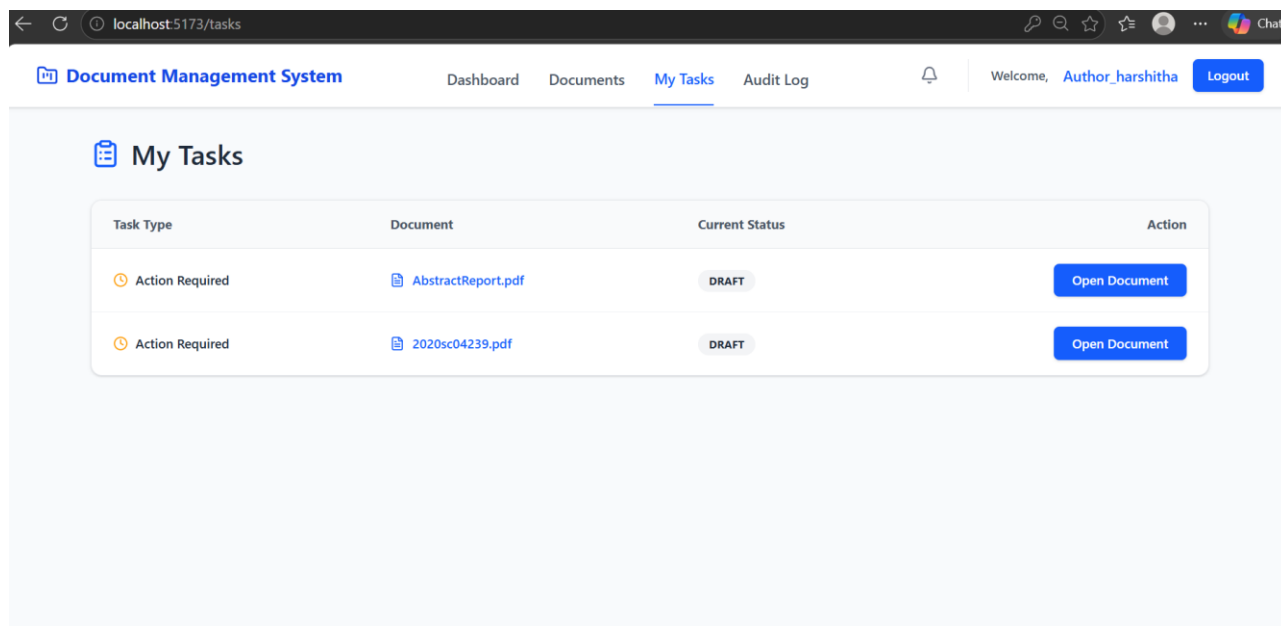
The above page shows Author's view while sending the document for review.



The above page shows the reviewer's view while approving or rejecting the document.

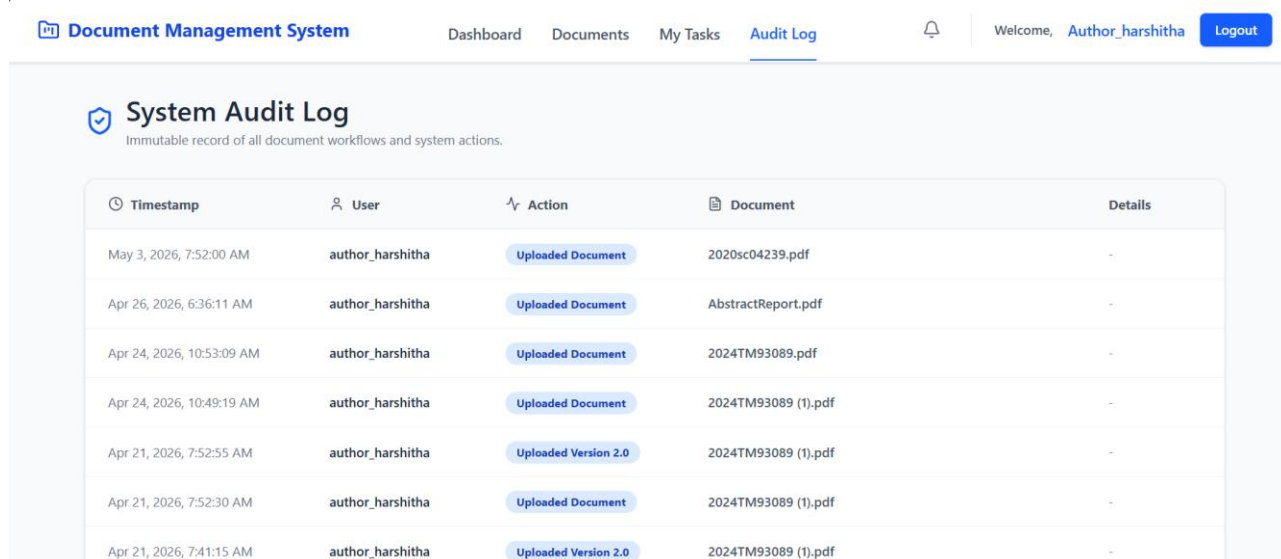
6.6 My Tasks Page

The My Tasks page provides a quick way for users to find their work. Depending on what their user type is, Authors will view their uploaded or submitted documents, while Reviewers will only see the documents assigned to them. Therefore, once on the My Tasks page, users will have instant access to the tasks they currently have outstanding and will not have to search the main document directory for their outstanding tasks.



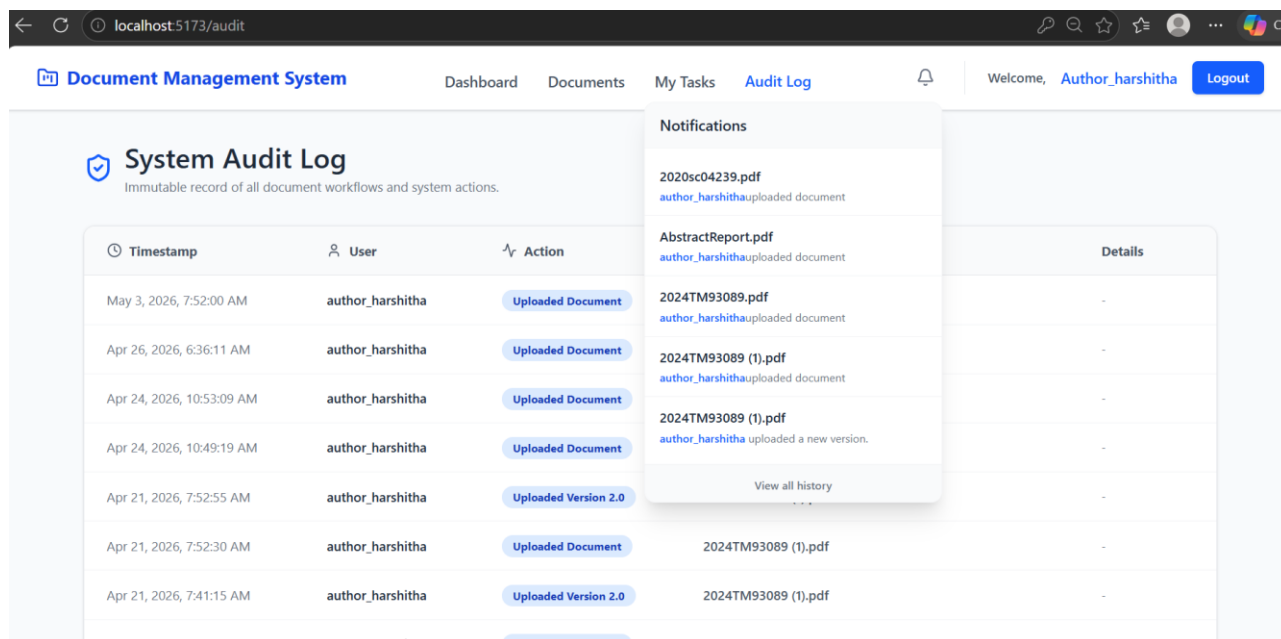
6.7 Audit Logs Page

The Audit Log page keeps a full and continuous log of every action that takes place within the application and documents how a document is modified. The page displays who performed the action, the time/date the action was performed, and what action was performed to provide complete accountability for all workflow events that occur.



6.8 Notifications Tab

The Notifications tab keeps users updated on important system events. Authors receive alerts when their documents are approved or rejected, including direct access to any rejection comments. Reviewers, on the other hand, are notified whenever a document is assigned to them for review. This feature ensures that all users remain informed and can quickly respond to workflow changes.



6.9 System Performance and Observations

The system demonstrates efficient automation and responsiveness under normal operating conditions. The APScheduler runs at fixed intervals of 60 seconds to monitor document deadlines and trigger state transitions.

Key observations:

- Automated task creation eliminates manual assignment delays.
- Real-time UI updates ensure users receive immediate feedback on actions.
- The system successfully handles concurrent operations using PostgreSQL without data conflicts.
- PDF processing is optimized with a limit of 3000 words to prevent performance degradation.

These observations confirm that the system achieves its objective of reducing manual effort while maintaining reliability and data integrity.

CHAPTER 7: TESTING

7.1 Testing Approach

The system was tested using both manual and API-based testing methods. Postman was used to validate REST API endpoints, while the frontend was tested through user interaction

scenarios.

7.2 Sample Test Cases

Test Case	Input	Expected Output	Result
Login	Valid credentials	Login Successful	Passed
Upload Document	PDF file/ Word Document	Document stored	Passed
Role Restriction	Author tries approval	Access denied	Passed
Deadline Trigger	Expired document	Automatically updated to Review	Passed
Task Assignment	Submit document	Task created	Passed

Table 3

7.3 Validation

All core functionalities were tested successfully, ensuring that the system behaves as expected under different user roles and workflow conditions.

CHAPTER 8: CONCLUSIONS

8.1 Summary

The project successfully conceptualized, designed and deployed a comprehensive Document Workflow Automation System. This system reduces many of the administrative bottlenecks for businesses that operate in highly regulated industries. By using an FSM (finite state machine) for the application rather than a legacy manual method (i.e. email-based tracking), the automated application is designed and built to be very reliable with data integrity.

With the use of APScheduler to implement background scheduling, the project was able to automate time-sensitive state transitions and task delegations to demonstrate that lightweight concurrency can be used effectively instead of manually overseeing many of these tasks. In addition, strict RBAC (role-based access control) was implemented using secure JWT (JSON Web Token) and a custom isolated PDF viewer in React elevated the overall security posture of the system. Overall, the successful delivery of a decoupled,

modern technology stack (React.js, FastAPI, PostgreSQL) provides a user-friendly solution that is highly transparent and efficient for managing complex document lifecycles. Besides meeting all of the functional requirements it also provides a scalable architecture for other workflow automation solution enhancements in the future.

8.2 Future Scope

While the current architecture successfully meets all defined objectives, the architecture has been designed and built to support modularity, which allows us to scale to an enterprise level in the future. Areas of future enhancement that are being considered include:

- **Distributed Task Queues:** Currently, highly efficient in-process scheduling mechanism for our current workload is being utilized. However, in order to achieve horizontal scaling across a large number of users, it would make sense to decouple the background processing architecture from the current type of processing architecture to create a distributed architecture that utilizes message brokers (like RabbitMQ, Redis, etc.) through asynchronous processing worker frameworks (like Celery).
- **Automated Database Migrations:** The use of migration tools (like Alembic) would improve the maintainability aspect of a database. They would allow for schema changes and/or column modifications in live environments without having to manually write SQL code to create or alter tables.
- **Expanded Format Support:** Future versions of the custom in-browser renderer may also be able to render in file formats such as Word (.docx) and Excel (.xlsx) without downloading a local file.
- **Advanced AI Integration:** Currently, the current extraction module (PyPDF2) could be enhanced by integrating with Large Language Models (LLMs) to provide automation of compliance checks, enhance semantic search functionality or allow for near-automated contextual summarization in addition to normal text extraction.

8.3 Challenges Faced:

- **Migrated the DB from SQLite to PostgreSQL:** While SQLite has many advantages while developing an application, it has performance issues with simultaneous writes from multiple sources. Changing to PostgreSQL was necessary to prevent the application from locking or crashing if an automated background task and a live user updated the status of an application at the same time.

- **Managing File Uploads with Metadata:** When sending a physical PDF file along with user information (like the deadline for uploading or title) from the user interface to the server, special care needed to be made to send the content. The standard JSON request does not send file data therefore, the front end needed to be modified to send the content using the multipart/form-data request format so special back-end processing would need to be written to parse the bytes of the file and the text file field.
- **Real-time UI Synchronization:** The front end (React application) needed to automatically receive back-end updates without user intervention (i.e., without or before the page is manually refreshed). For instance, when the reviewer clicks the approve link, the front end needs to programmatically re-fetch the document status from the back-end API so the status can be updated dynamically (e.g., from "In Review" to "Approved").

REFERENCES

1. Sommerville, I., Software Engineering, Pearson Education.
2. Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.
3. Fowler, M., Patterns of Enterprise Application Architecture, Addison-Wesley.
4. W3C, RESTful Web Services Architecture, W3C Recommendations.
5. Richards, M., & Ford, N., Fundamentals of Software Architecture, O'Reilly Media.
6. IEEE, Software Architecture and Design Standards, IEEE Publication.
7. van der Aalst, W. M. P., & van Hee, K. M., Workflow Management: Models, Methods, and Systems, MIT Press.
8. Jones, M., Bradley, J., & Sakimura, N., *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force (IETF).
9. Ramalho, L., *Fluent Python: Clear, Concise, and Effective Programming*, O'Reilly Media.
10. Banks, A., & Porcello, E., *Learning React: Modern Patterns for Developing React Apps*, O'Reilly Media.
11. Newman, S., *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media.

ABBREVIATIONS

- **API:** Application Programming Interface
- **RBAC:** Role-Based Access Control
- **CRUD:** Create, Read, Update, Delete
- **JSON:** JavaScript Object Notation

- **JWT:** Java Web Tokens
- **PDF:** Portable Document Format

APPENDIX

GitHub Link: [Link to the Code](#)

Checklist of Items for the Final Dissertation / Project / Project Work Report

This checklist is to be attached as the last page of the final report.

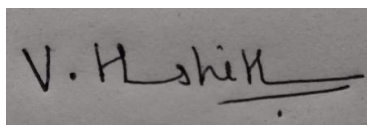
This checklist is to be duly completed, verified and signed by the student.

1.	Is the final report neatly formatted with all the elements required for a technical Report?	Yes
2.	Is the Cover page in proper format as given in Annexure A?	Yes
3.	Is the Title page (Inner cover page) in proper format?	Yes
4.	(a) Is the Certificate from the Supervisor in proper format? (b) Has it been signed by the Supervisor?	Yes Yes
5.	Is the Abstract included in the report properly written within one page? Have the technical keywords been specified properly?	Yes Yes
6.	Is the title of your report appropriate? The title should be adequately descriptive, precise and must reflect scope of the actual work done. Uncommon abbreviations / Acronyms should not be used in the title	Yes
7.	Have you included the List of abbreviations / Acronyms?	Yes
8.	Does the Report contain a summary of the literature survey?	Yes
9.	Does the Table of Contents include page numbers? i. Are the Pages numbered properly? (Ch. 1 should start on Page # 1) ii. Are the Figures numbered properly? (Figure Numbers and Figure Titles should be at the bottom of the figures) iii. Are the Tables numbered properly? (Table Numbers and Table Titles should be at the top of the tables) iv. Are the Captions for the Figures and Tables proper? v. Are the Appendices numbered properly? Are their titles appropriate	Yes Yes Yes Yes Yes
10.	Is the conclusion of the Report based on discussion of the	Yes

	work?	
11.	Are References or Bibliography given at the end of the Report? Have the References been cited properly inside the text of the Report? Are all the references cited in the body of the report	Yes Yes
12.	Is the report format and content according to the guidelines? The report should not be a mere printout of a PowerPoint Presentation, or a user manual. Source code of software need not be included in the report.	Yes

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.



Place: Bengaluru

Date: 10-May-2026

Signature of the Student

Name: V HARSHITHA

ID No.: 2024TM93089